Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Classes e Objetos

Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br

Roteiro

- Objetos
- Classes
- Implementando Classes
- Manipulando Objetos
- Tempo de Vida

Objeto

- **Relembrando…**
  - Entidade que existe no tempo e no espaço

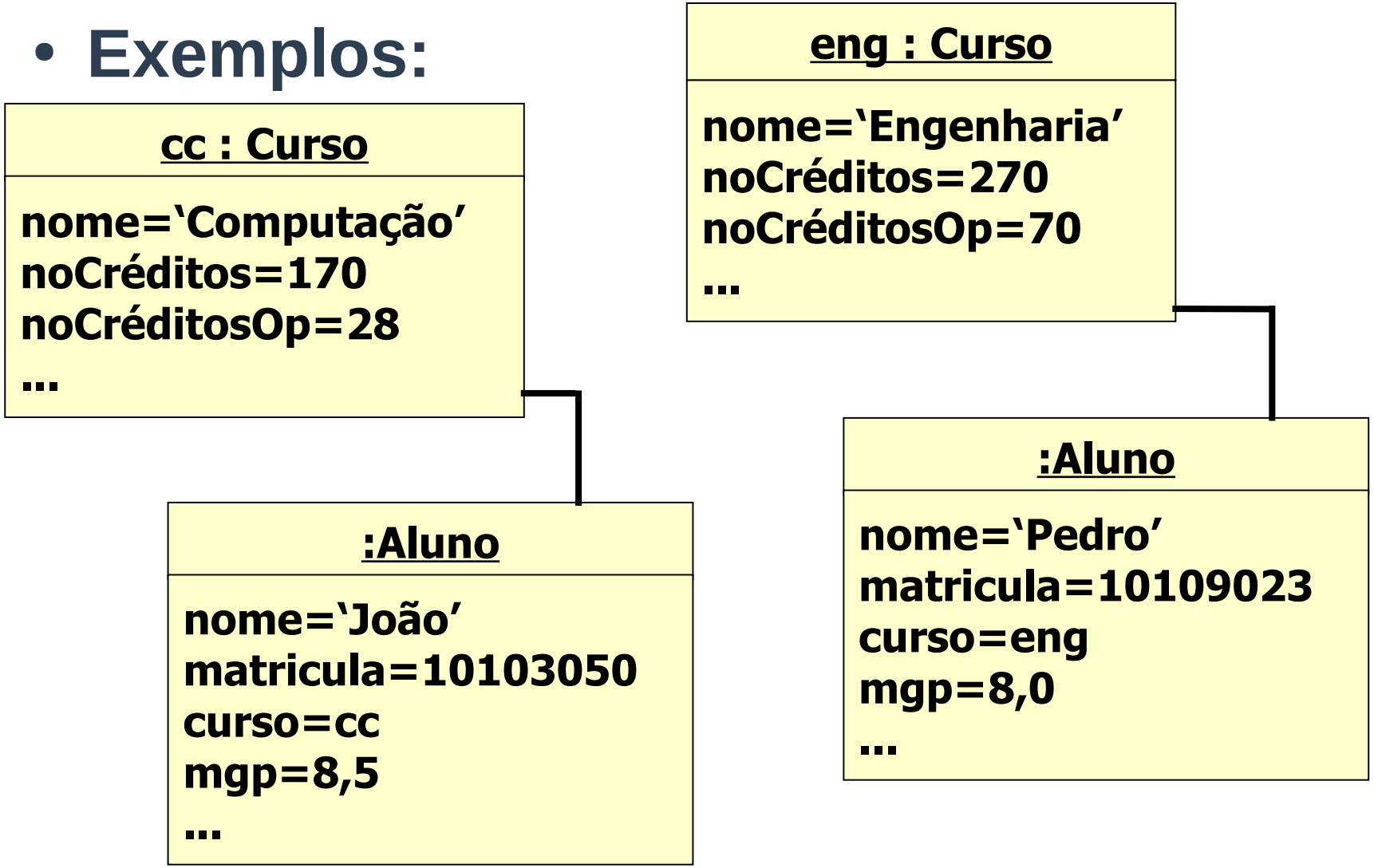
- **Exemplo de objetos**
  - Aluno
  - Professor
  - Turma

Objetos

- **Objetos são descritos por suas características**
  - Como podemos descrever
    - Um curso?
    - Um aluno?
    - Uma turma?

Objetos

- **Exemplos:**



Objetos

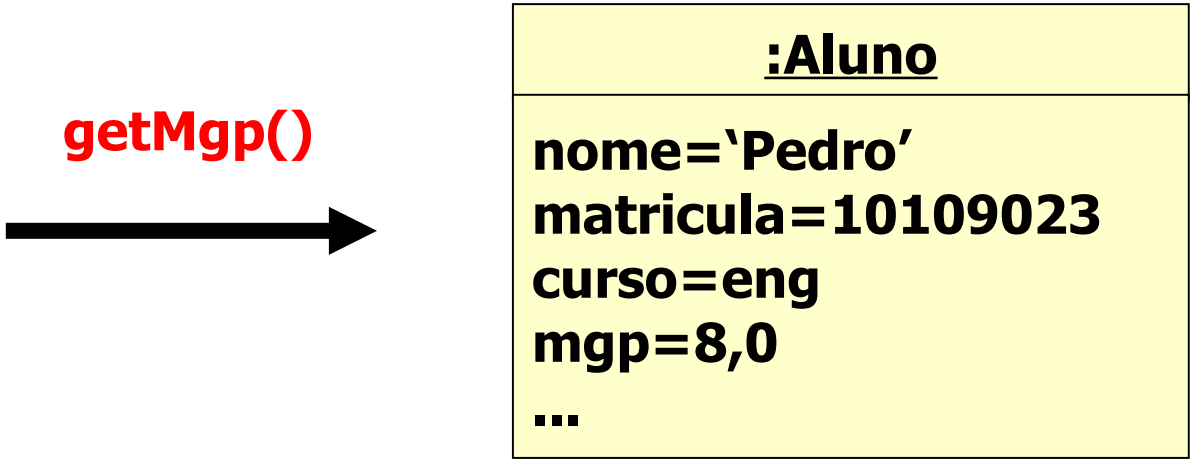
- **Operações que podem ser realizadas sobre os objetos**

matricularEmCurso(10103078, cc)

:Aluno
nome=‘Carolina’ **matricula=** **curso=** **mgp=** **...**

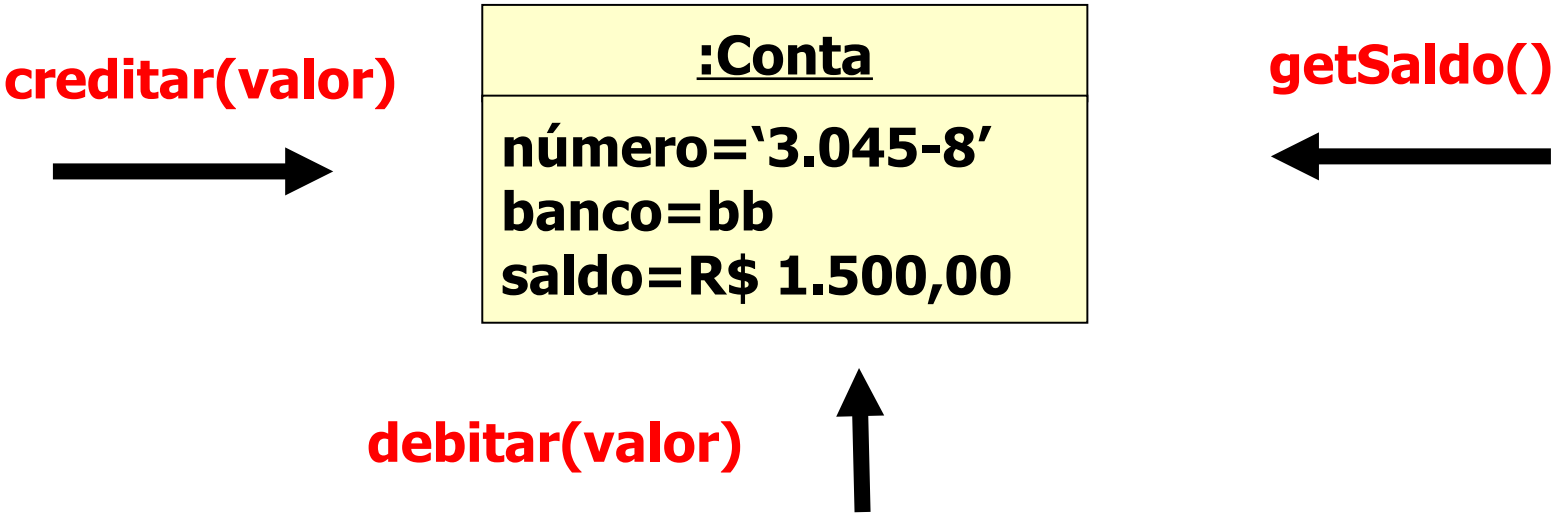
Objetos

- **Operações que podem ser realizadas sobre os objetos**



Objetos

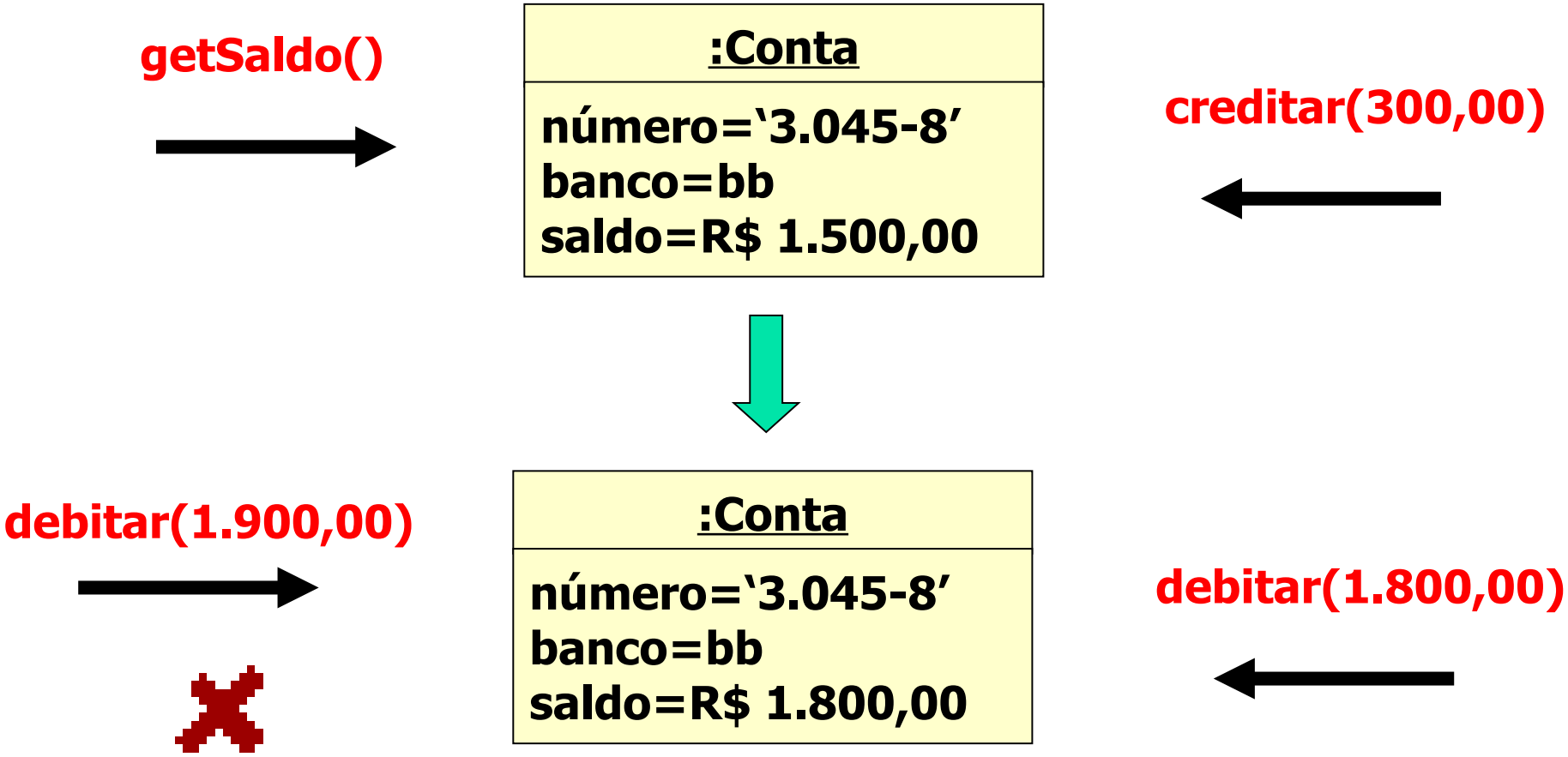
- **Outro exemplo**



  - Suas propriedades ou campos
    - Formam o **estado do objeto**
  - O **comportamento** do objeto é definido pelo seu estado e operações possíveis

Objetos

- Qual o comportamento do objeto?

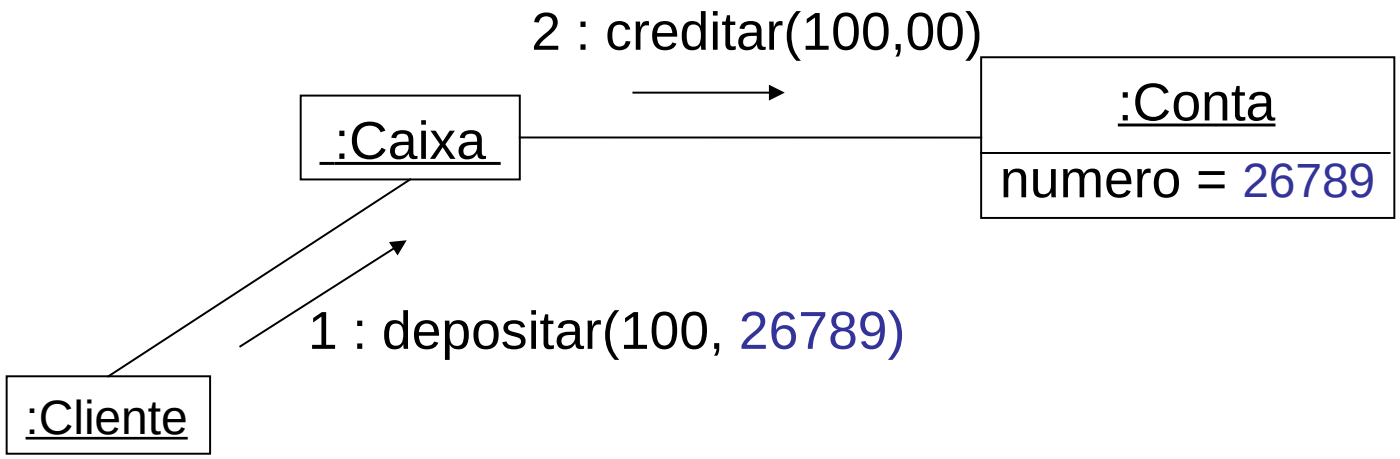


Objetos

Objeto = Estado + Comportamento + Identidade

Objetos

- **Objetos interagem por meio de troca de mensagens**
- **Por que é importante a troca de mensagens?**
  - Interação = colaboração
  - Colaboração → ações

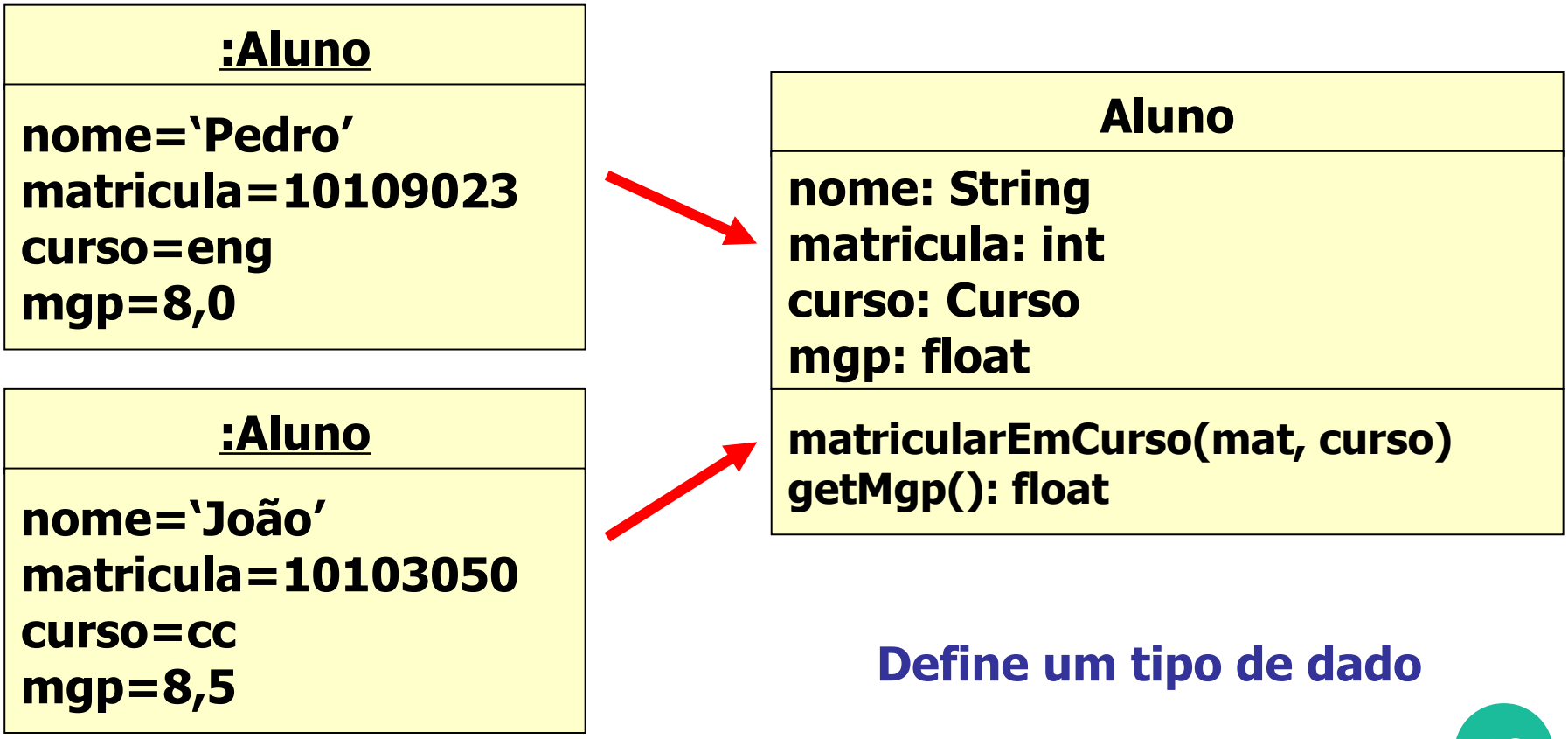


Classes

- **A classe é um *template* (molde/padrão) que permite criar objetos (instâncias)**
- **Uma classe descreve um grupo de objetos**
  - Propriedades semelhantes
  - Comportamentos semelhantes
  - Relacionamentos comuns com outros objetos

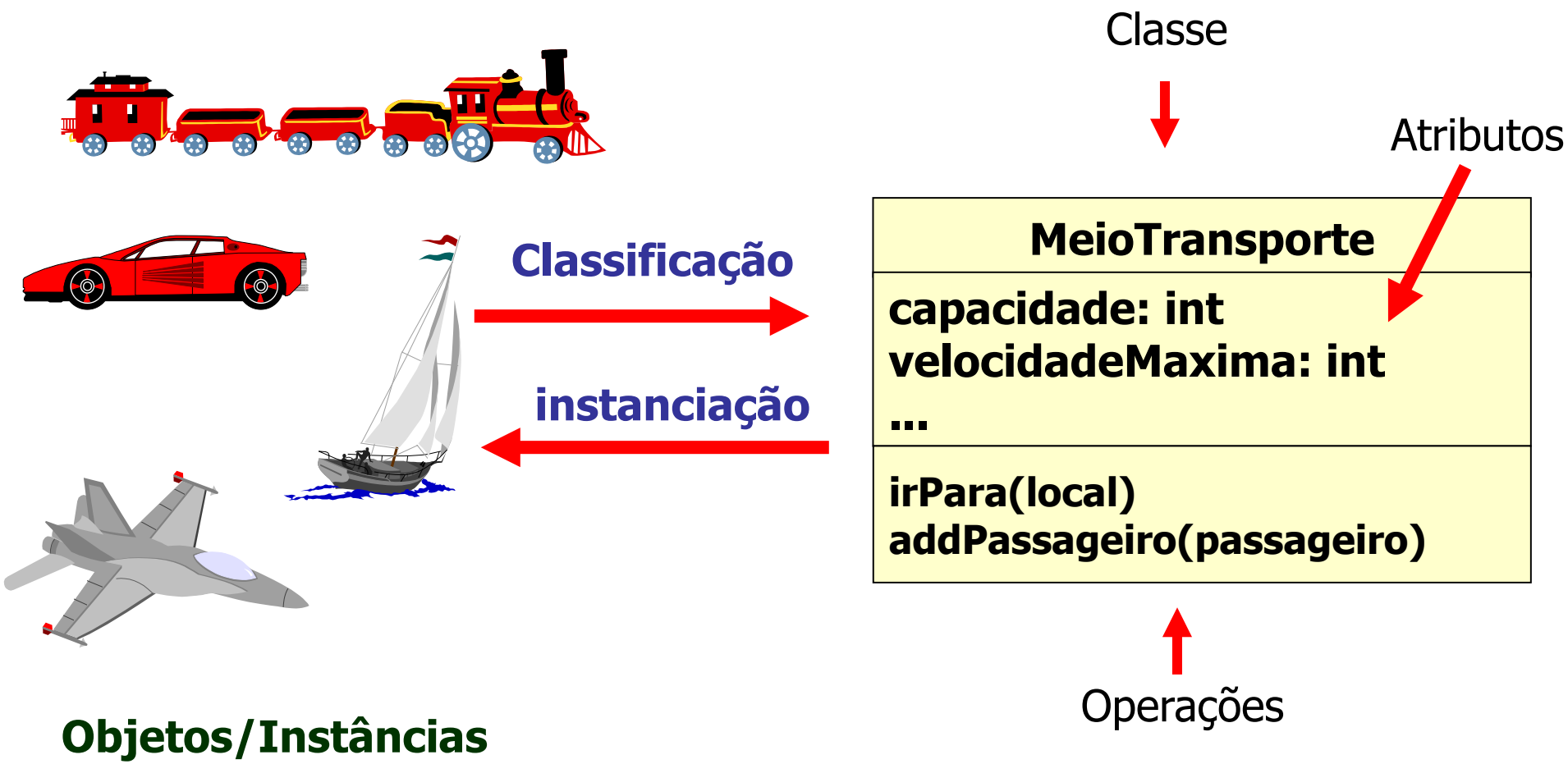
Classes

- Abstração que representa a estrutura e o comportamento comum



Define um tipo de dado

Classes


Classe
Atributos
MeioTransporte
capacidade: int
velocidadeMaxima: int
...
irPara(local)
addPassageiro(passageiro)
Operações
Classificação
instanciação
Objetos/Instâncias

Classes

- **Atributos**
  - São as propriedades nomeadas de um objeto
    - Conta bancária
      - Saldo, titular, data de abertura, etc.
    - Pessoa
      - Nome endereço, telefone, etc.
    - Livro
      - isbn, nome, autor, edição, etc.

Classes

- **Operações**
  - Definem o comportamento possível para um objeto
    - Correspondem a ações a serem executadas
  - Nomenclatura utilizada
    - Chamada de método
    - Chamada de função
    - Passar uma mensagem

Classes

- **Teoricamente**
  - Operação <> método
    - Operação = especificação da ação (assinatura)

```
public static void soma(int num1, int num2);
```

    - Método = implementação da ação

```
public static void soma(int num1, int num2) {
        int resultado = num1 + num2;
        System.out.println("Soma = " + resultado);
  }
```

Classes

- **Detalhes**
  - Exemplos de passagem de parâmetro e retorno

Classes

```
public class Teste1 {
    public static void main(String[] args) {
        soma();     // sem a passagem de parâmetro e sem retorno
    }
    public static void soma() {
            Scanner e = new Scanner (System.in);
            int num1, num2, resultado;
            System.out.println("Digite o primeiro número:");
            num1 = e.nextInt();
            System.out.println("Digite o segundo número:");
            num2 = e.nextInt();
            resultado = num1 + num2;
            System.out.println("Soma = " + resultado);
   }
}
```

Classes

```
public class Teste2 {
   public static void main(String[] args) {
        Scanner e = new Scanner (System.in);
        int num1, num2;
        System.out.println("Digite o primeiro número:");
        num1 = e.nextInt();
        System.out.println("Digite o segundo número:");
        num2 = e.nextInt();
        soma(num1, num2);        //com parâmetros e sem retorno
    }
    public static void soma(int num1, int num2) {
        int resultado = num1 + num2;
        System.out.println("Soma = " + resultado);
    }
}
```

Classes

```
public class Teste3 {
   public static void main(String[] args) {
         int resultado = soma();        //sem parâmetro e com retorno
        System.out.println("Soma = " + resultado);
   }
   public static int soma() {
       Scanner e = new Scanner (System.in);
      int num1, num2;
      System.out.println("Digite o primeiro número:");
       num1 = e.nextInt();
       System.out.println("Digite o segundo número:");
       num2 = e.nextInt();
       return num1 + num2;
   }
}
```

Classes

```
public class Teste4 {
    public static void main(String[] args) {
        Scanner e = new Scanner (System.in);
        int num1, num2;
        System.out.println("Digite o primeiro número:");
        num1 = e.nextInt();
        System.out.println("Digite o segundo número:");
        num2 = e.nextInt();
        int resultado = soma(num1, num2);   //com parâmetro e com retorno
        System.out.println("Soma = " + resultado);
    }
    public static int soma(int num1, int num2) {
            return num1 + num2;
    }
}
```

Implementando Classes

Produto
codigo: int nome: String quantidade: int preco:float
Produto (int codigo, String nome, int quantidade, float preco) getCodigo(): int getNome(): String getQuantidade(): int getPreco(): float setCodigo(int codigo) setNome(String nome) setQuantidade(int quantidade) setPreco(float preco) obterValorEstoque(): float

Implementando Classes

```
public class Produto {

   /* Atributos da Classe */

   private int codigo;
   private String nome;
   private int quantidade;
   private float preco;
```

Implementando Classes

```
/* construtor da classe */

public Produto (int codigo, String nome, int quantidade, float preco) {

    this.codigo = codigo;
    this.nome = nome;
    this.quantidade = (quantidade > 0)?  quantidade:0;
    this.preco = preco;

}
```

Implementando Classes

- **Construtor** é o método da classe que é executado quando deseja-se **criar** uma **instância** da classe (objeto)
- Responsável por **alocar** recursos
- Permite **inicializar** os atributos do objeto
- Toda classe deve possuir um construtor
- O nome do construtor é o **mesmo nome da classe** e **não possui retorno**

Implementando Classes

- **Cada objeto tem acesso a uma referência para si próprio, através de this**
- **Através de this é possível:**
  - Diferenciar entre parâmetros de métodos e atributos de instância com o mesmo nome
  - Retornar uma referência para o próprio objeto em um de seus métodos

Implementando Classes

- **O método de acesso é a única forma de ter acesso aos dados internos (privados)**
- **Protegem a representação dos dados**

```
/* métodos de acesso */

public int getCodigo() {
   return this.codigo;
}

public String getNome() {
   return this.nome;
}

public int getQuantidade() {
   return this.quantidade;
}

public float getPreco() {
   return this.preco;
}
```

Implementando Classes

- **O método modificador ou mutante permite alterar o estado interno de um objeto**

```
/* métodos modificadores ou mutantes */

public void setCodigo(int codigo) {
        this.codigo = codigo;
}

public void setNome(String nome) {
        this.nome = nome;
}

public void setQuantidade(int quantidade){
        this.quantidade = (quantidade > 0)?  quantidade:0;
}

public void setPreco(float preco) {
        this.preco = preco;
}
```

Implementando Classes

- **Métodos do negócio são necessários para implementar as funcionalidades requeridas pela aplicação**

```
/* método do negócio */

public float obterValorEstoque() {
   return this.preco * this.quantidade;
}
```

Manipulando Objetos

- **O que faz esse código?**

Declara a variável p1

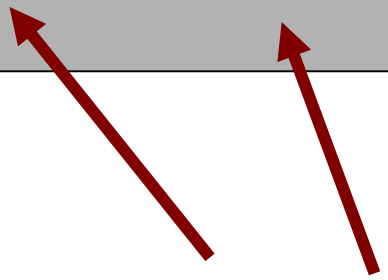
```
Produto p1 ;
p1 = new Produto(1, "Resma Papel", 10, 14.5);
```

New cria uma instância de Produto a partir de seu construtor

Manipulando Objetos

- **O que faz esse código?**

```
Produto p2 = new Produto(2, "Lápis", 15, 1.0);
```

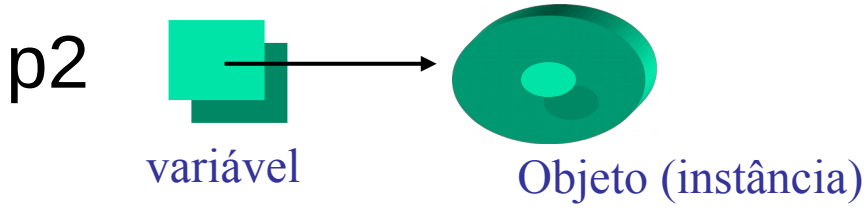


Declaração + Inicialização

Manipulando Objetos

- **Importante**

```
Produto p2 = new Produto(2, "Lápis", 15, 1.0);
```



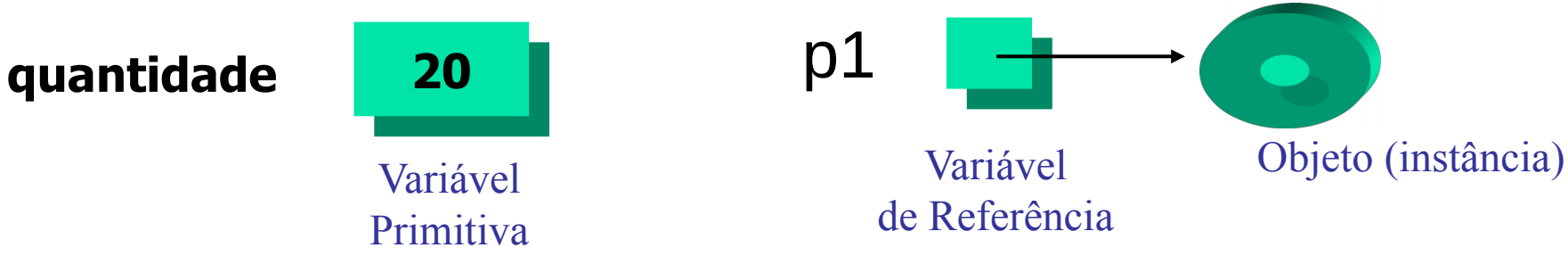
- **Em Java**
  - Uma variável não é (não contém) um objeto
  - Uma variável contém uma referência a um objeto

Manipulando Objetos

- **Variável Primitiva x Variável de Referência**
  - Variável primitiva → um valor
  - Variável de Referência → como chegar a um objeto

```
int quantidade = 20;
```

```
Produto p1 = new Produto();
```



Manipulando Objetos

- **Declaração, criação e atribuição de objetos**

Declara a variável de referência p1

```
Produto p1 ;

p1 = new Produto(1, "Resma Papel", 10, 14.5);
```

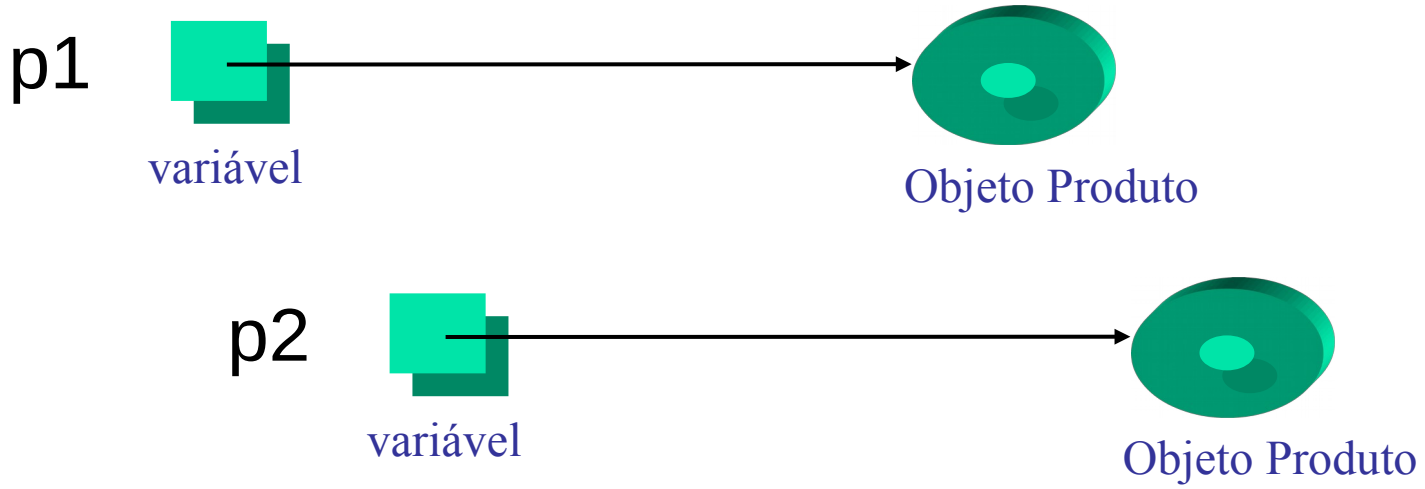
New cria uma instância de Produto a partir de seu construtor

Vincula objeto à referência

Manipulando Objetos

- **Referências**

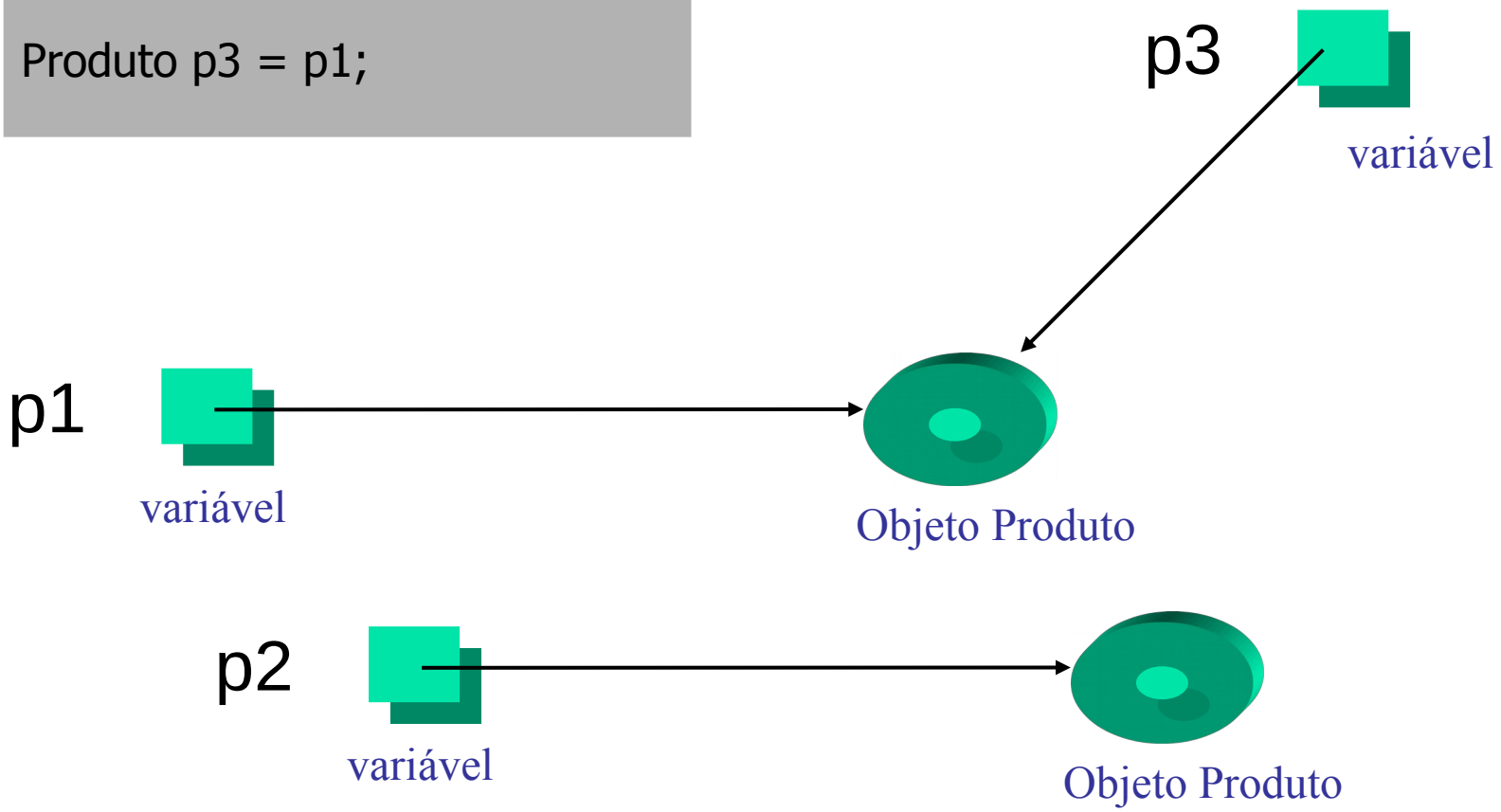
```
Produto p1 = new Produto(1, "Resma Papel", 10, 14.5f);

Produto p2 = new Produto(2, "Lápis", 15, 1.0f);
```



Manipulando Objetos

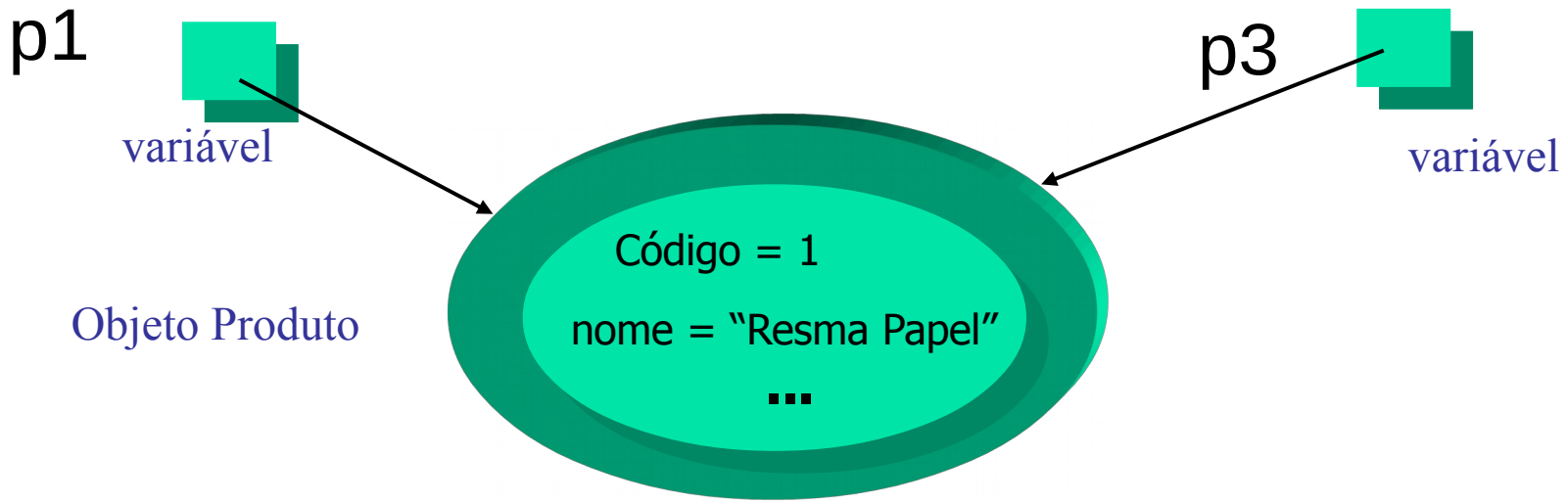
- **p3 referencia qual objeto?**

```
Produto p3 = p1;
```



Manipulando Objetos

- **Então...**



Fazendo...

```
p3.setNome("Papel");
```

Qual o resultado?

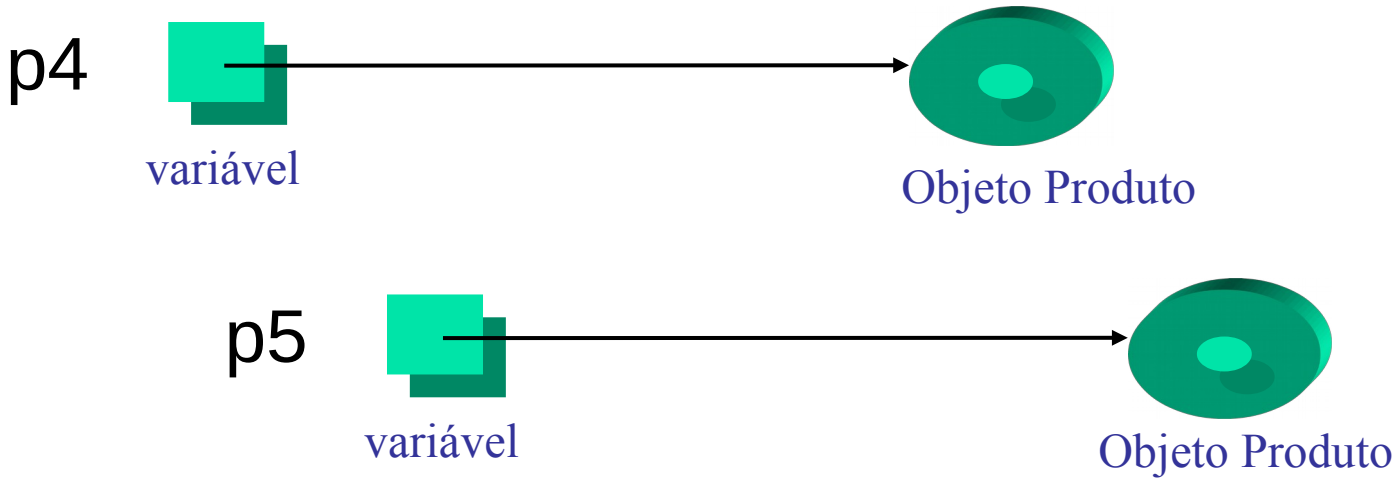
```
? = p1.getNome();
```

Manipulando Objetos

- **Outro Exemplo**

```
Produto p4 = new Produto(3, "Borracha", 15, 2.5f);

Produto p5 = new Produto(3, "Borracha", 15, 2.5f);
```

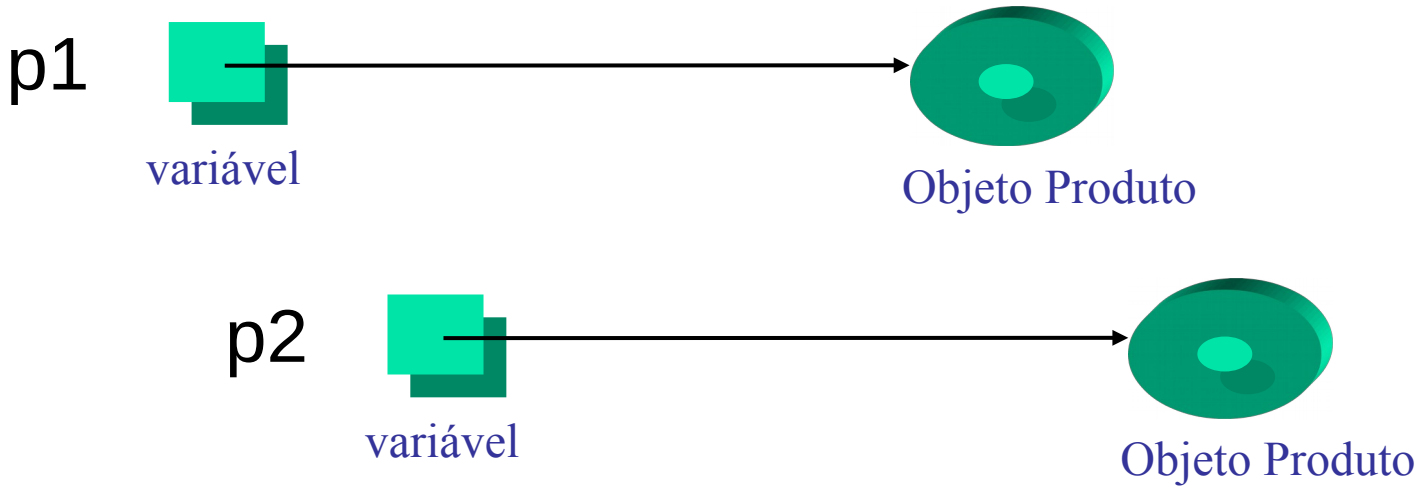
p4 == p5 ? ✗



Manipulando Objetos

- **Outro Exemplo**

```
Produto p1 = new Produto(1, "Resma Papel", 10, 14.5f);

Produto p2 = new Produto(2, "Lápis", 15, 1.0f);
```



Manipulando Objetos

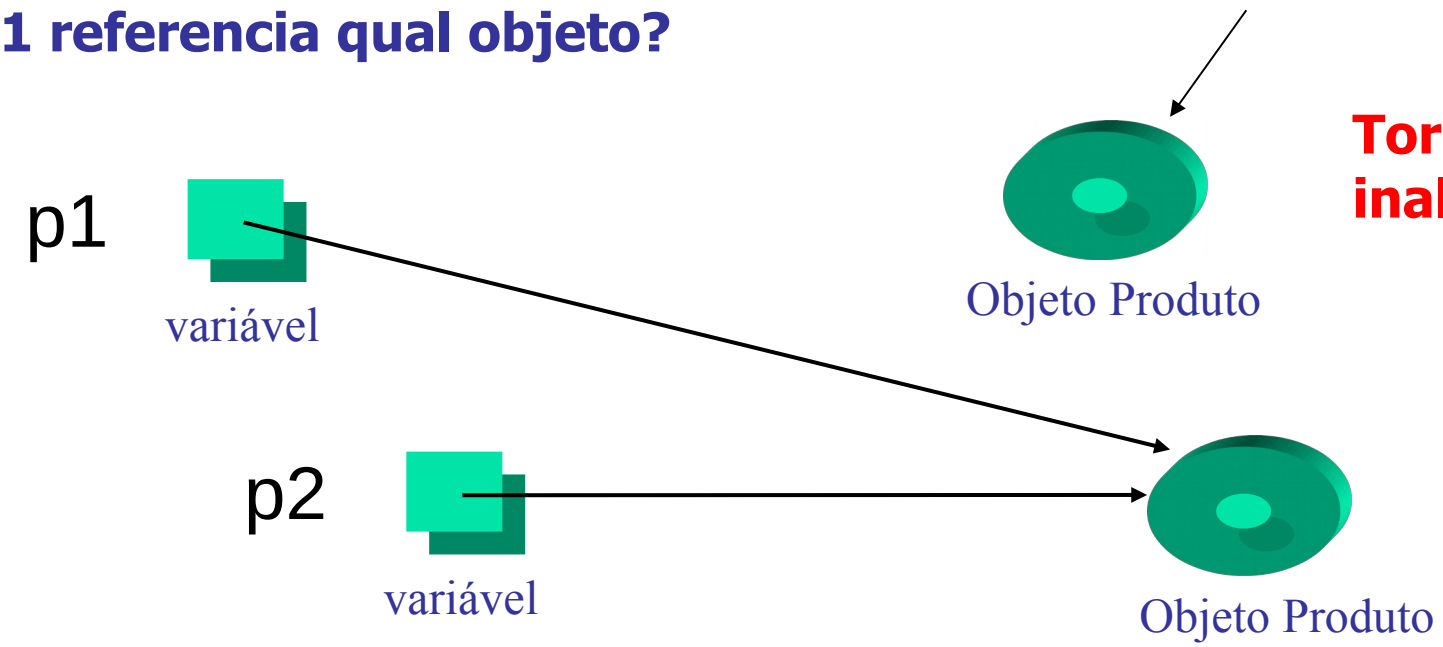
- **Outro Exemplo**

```
p1 = p2;
```

P1 referencia qual objeto?

O que acontece com este objeto?

Torna-se inalcançável



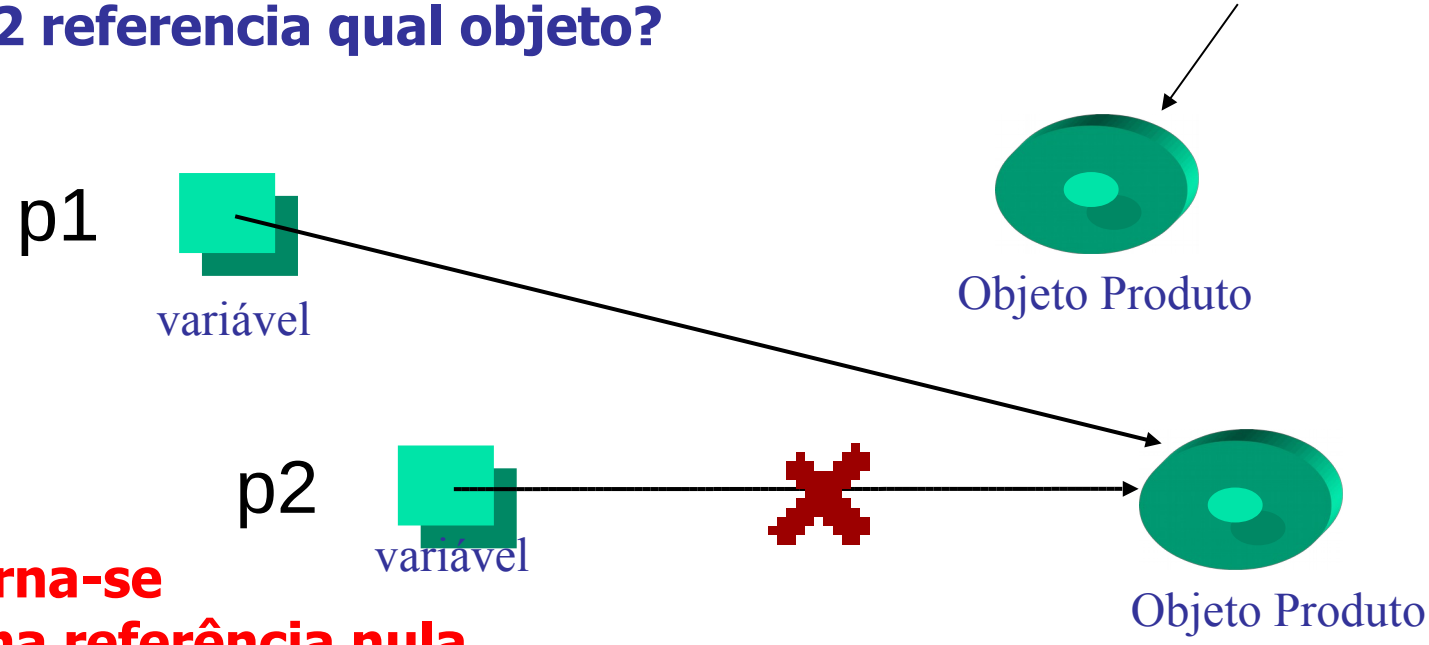
Manipulando Objetos

- **Se fizermos**

```
p2 = null;
```

Objeto anteriormente abandonado

P2 referencia qual objeto?

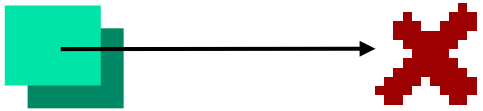


Torna-se
Uma referência nula

Manipulando Objetos

- **Null**
  - Indica uma referência para nenhum objeto

```
Produto p6 = null;
```



Variável de Referência

  - Qualquer objeto do tipo definido para a referência pode ser vinculado à mesma

```
p6 = new Produto();
```

Manipulando Objetos

- **Null**

```
Produto p7 = null;
```

```
p7.getNome();
```

NullPointerException

Manipulando Objetos

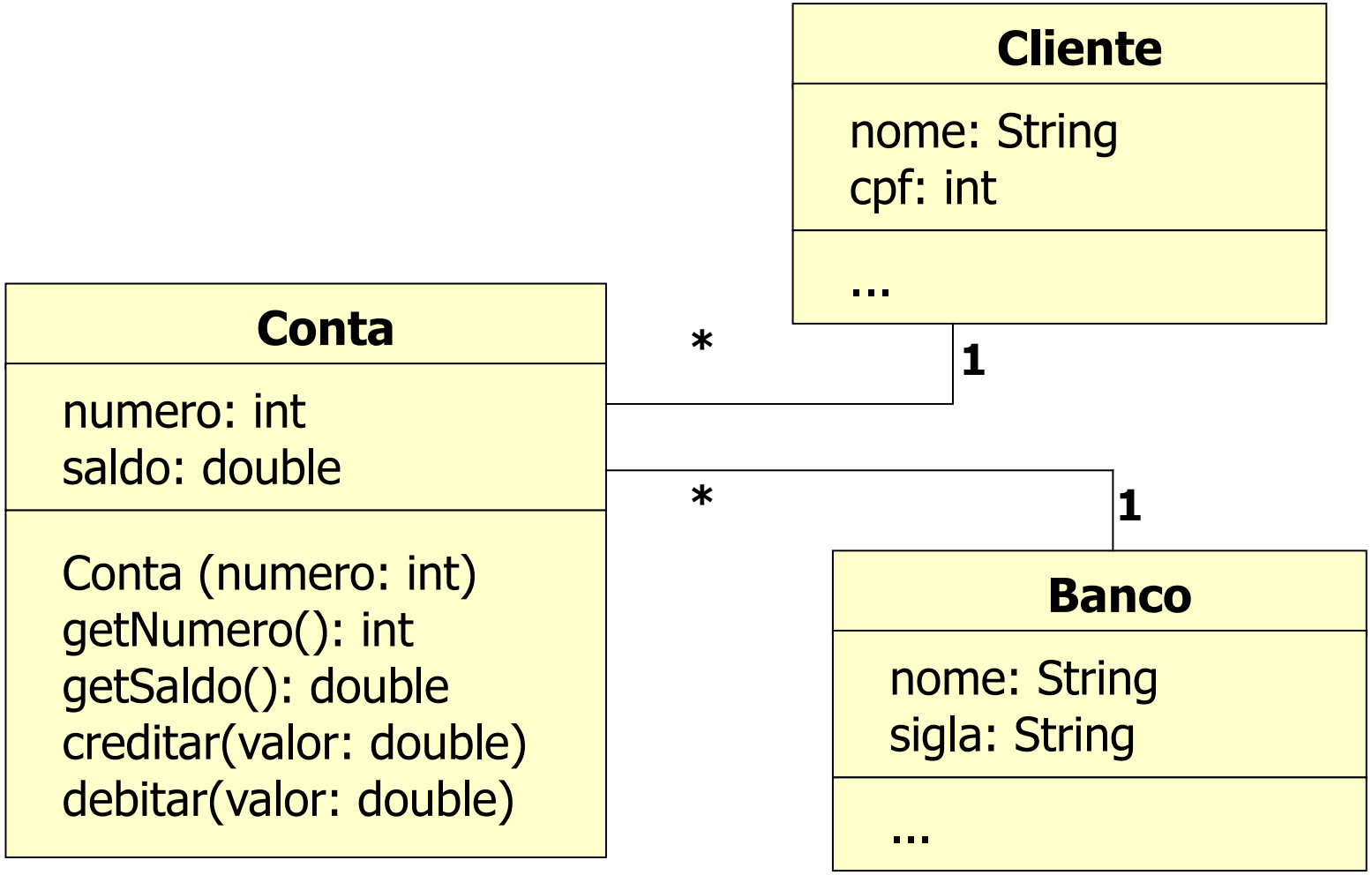
- **Inicialização de variáveis**
  - Variáveis de instância do tipo primitivo são automaticamente inicializadas pelo default
    - Boolean → false
    - Demais tipos → 0
  - Variáveis locais ou temporárias não são automaticamente inicializadas por default
  - Variáveis por referência não são automaticamente inicializadas por default

Implementando Classes

Conta
numero: int saldo: double
Conta (numero: int) getNumero(): int getSaldo(): double creditar(valor: double) debitar(valor: double)

```
public class Conta {
   private int  numero;
   private double   saldo;
   public Conta (int num) {
         numero = num; saldo = 0;
     }
     public int getNumero() {return numero;}
     public double getSaldo() {return saldo;}
     public void creditar (double valor) {
         saldo = saldo + valor;
     }
   public void debitar (double valor) {
         int novoSaldo = saldo - valor;
         if (novoSaldo >= 0) saldo = novoSaldo;
     }
   }
```

Implementando Classes



Implementando Classes

- **Novas variáveis de instância são criadas**

```
public class Conta {

private int  numero;
private double   saldo;

private Cliente titular;
private Banco banco;
...
```

Implementando Classes

```
public class Conta {
   ...
public Cliente getTitular( ) {
            return this.titular;
   }

   public Banco getBanco( ) {
            return this.banco;
   }

public setTitular(Cliente cliente) {
            this.titular = cliente;
   }

   public setBanco(Banco banco) {
         this.banco = banco;
  }
...
```

Implementando Classes

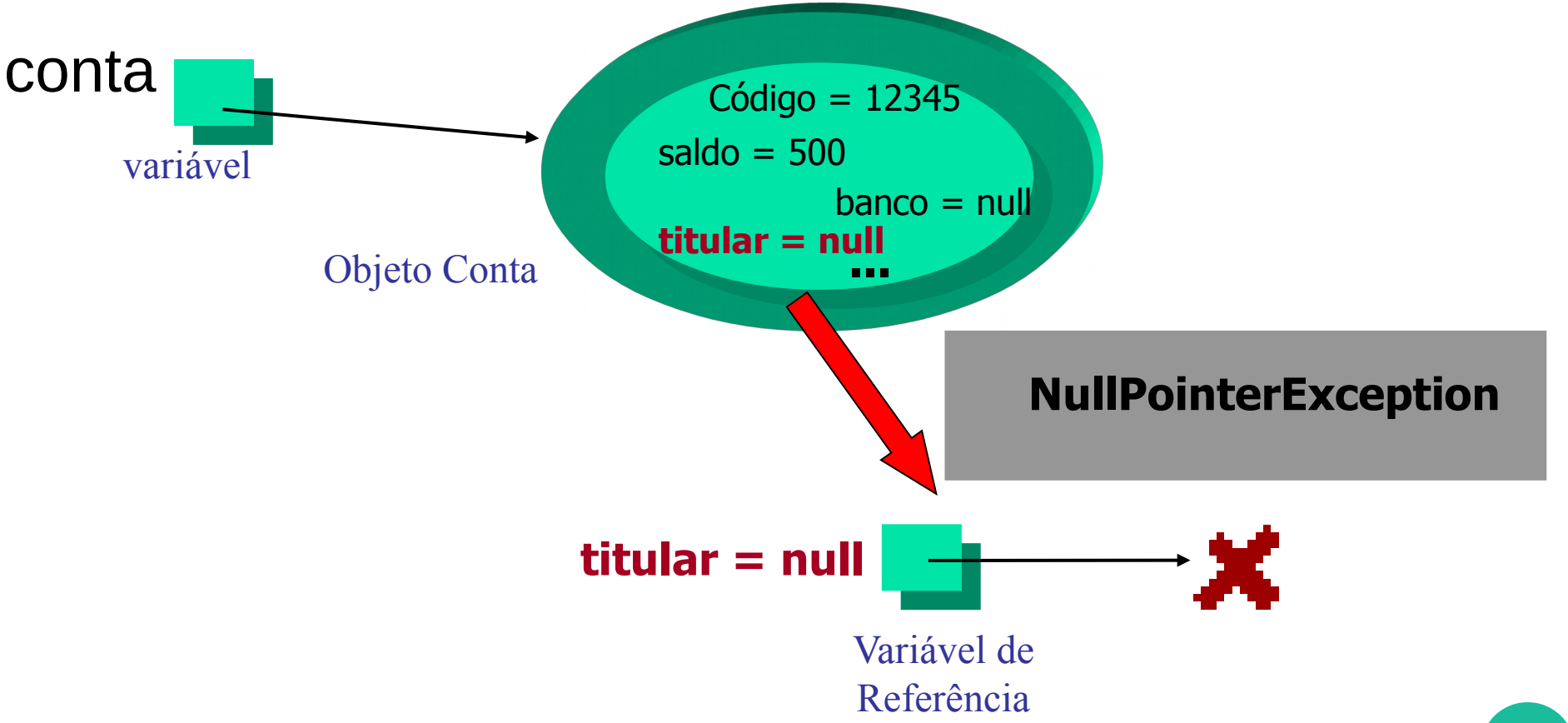
- **O que acontece ao implementar esse código?**

```
Conta conta = new Conta(12345);

conta.creditar(500);

System.out.println(conta.getCliente().getNome());
```

- **Por que não funcionou?**

Manipulando Objetos

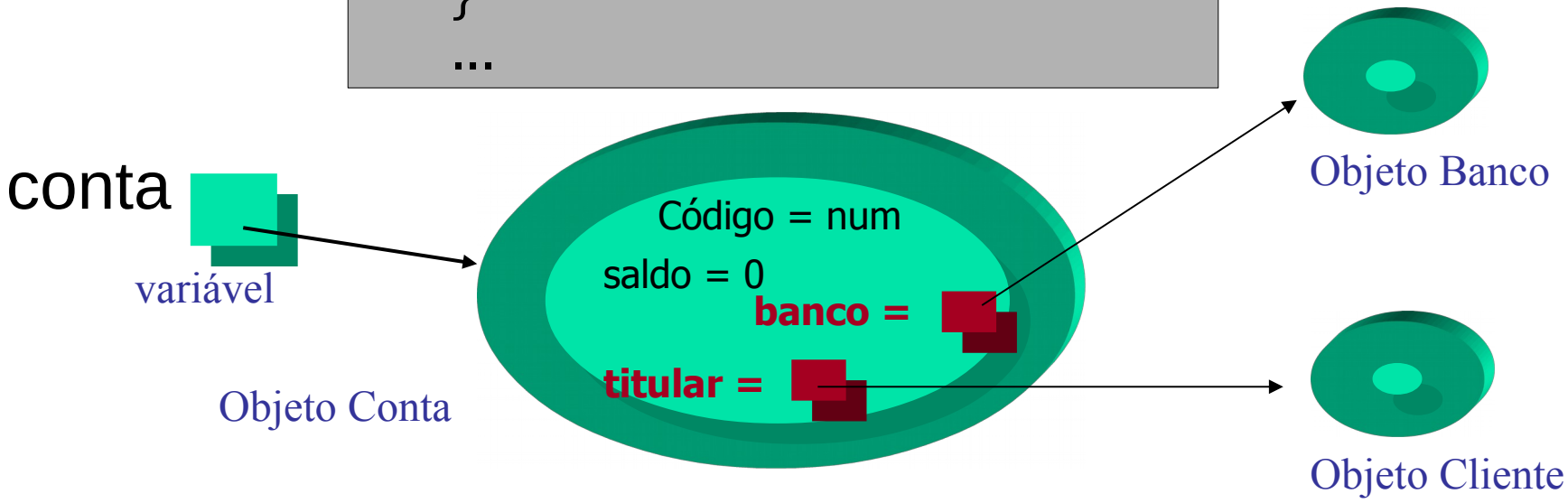
- **Qual a situação atual do objeto conta?**



Manipulando Objetos

- **Qual uma possível solução?**

```
public class Conta {
    ...
  public Conta (int num) {
      numero = num; saldo = 0;
      this.banco = new Banco( );
      this.titular = new Cliente( );
  }
  ...
```



Tempo de Vida

- A criação dos objetos é dinâmica (através do new)
- O tempo de vida de um objeto termina quando não existe mais nenhuma referência para ele
- Objetos cujos tempos de vida terminam são eliminados automaticamente (garbage collected)

Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Classes e Objetos

Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br